



BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 212
Numerical Techniques Laboratory

Final Project Report
Section: C2 Group: 04

DC Motor Dynamic Simulator

Course Instructors:
Nafis Sadik, Assistant Professor
Erom Hasan Niha, Adjunct Lecturer

Student ID	Name
2406177	Shuvro Pain
2406188	Taseen Intisar
2406193	Abdus Samiul Hasan Sun
2406195	Nadman Wasit

Contents

1	Abstract	2
2	Introduction	2
3	Design	3
3.1	Problem Formulation (PO(b))	3
3.1.1	Identification of Scope	3
3.1.2	Literature Review	3
3.1.3	Formulation of Problem	4
3.1.4	Analysis	4
3.2	Design Method	5
3.3	Simulation Model	6
3.4	Full Source Code	6
4	Implementation	8
4.1	Description of the Application	8
4.2	Baseline: Required Features Only	8
4.2.1	Electrical Response	8
4.2.2	Mechanical Response	8
4.2.3	Torque-Speed Characteristic	8
4.2.4	Comparison of the Two Numerical Methods	9
4.2.5	Time-Step Study	9
4.3	Optional Features, Demonstrated Individually	11
4.3.1	Armature Current Limiting	12
4.3.2	Thermal Model	12
4.3.3	Coulomb Friction	13
4.3.4	Variable Load-Torque Characteristics	13
4.3.5	Adaptive-Step Integration (RK45)	15
4.3.6	Motor-Shaft Animation	15
4.3.7	Comparison with Measured Motor Data	15
4.3.8	Summary of Optional Features	16
5	Design Analysis and Evaluation	16
5.1	Additional Features / Extensions	16
5.2	Investigations (PO(d))	17
5.2.1	Design of Experiment	17
5.2.2	Data Collection	17
5.2.3	Results and Analysis	17
5.2.4	Interpretation and Conclusions on Data	19
5.3	Limitations of Tools (PO(e))	19
6	Source Code Repository	19
7	Future Work	20
8	References	20

1 Abstract

This project presents the design and implementation of a MATLAB App Designer application that simulates the coupled electrical and mechanical dynamics of an armature-controlled DC motor. The motor is modeled as a third-order system in armature current i_a , angular speed ω , and angular position θ , with optional extensions for winding temperature, current limiting, and Coulomb friction. Two numerical integration methods – Forward Euler and fourth-order Runge–Kutta (RK4) – are implemented from first principles and are compared against a high-accuracy `ode45` reference solution used solely for validation. The application exposes all motor and simulation parameters through a graphical interface, provides step, ramp, pulse and user-imported voltage waveforms, and supports load-torque profiles that follow realistic torque–speed characteristics (constant, linear, quadratic, and constant-power loads). A time-step convergence study confirms that Forward Euler and RK4 achieve their expected first- and fourth-order global truncation error, respectively, once a transient-alignment error in the RK4 stage evaluation is corrected. Four representative test cases are simulated and analyzed, covering step, ramp, pulse and imported-waveform excitation, and the results are shown to agree with the analytically predicted steady-state operating points to within numerical tolerance.

2 Introduction

DC motors remain one of the most widely used actuators in industrial drives, robotics, electric vehicles, positioning systems, pumps, and laboratory automation, owing to their simple control characteristics and well-understood dynamic behavior. Because the electrical and mechanical subsystems of a DC motor are coupled through the back-EMF and the electromagnetic torque, its transient response cannot in general be solved in closed form once nonlinear elements such as current limiting, Coulomb friction, or a speed-dependent load are introduced. This makes the DC motor a natural and pedagogically useful example of a *complex engineering problem*: a system whose governing equations are simple to state individually (Kirchhoff’s voltage law for the armature circuit and Newton’s second law for rotation) but whose overall behavior requires numerical integration to predict, requires the engineer to choose and justify a numerical method against competing constraints of accuracy, stability, and computational cost, and has no single “correct” solution technique – Forward Euler, RK4, and adaptive methods each represent a different trade-off.

The objective of this project is therefore twofold: (i) to formulate the coupled differential-algebraic model of an armature-controlled DC motor from physical first principles, and (ii) to implement and compare at least two independent numerical integration schemes for solving it, packaged inside an interactive MATLAB App Designer application. Two alternative solution strategies were considered during design. The first, which was adopted, is to implement Forward Euler and RK4 as hand-written fixed-step integrators operating on an explicit state-derivative function, with `ode45` retained only as an independent, non-fixed-step reference for error estimation – this is required by the project’s function-usage restrictions. The second, rejected, alternative would have been to build the motor as a block diagram in Simulink; this was excluded both because it is explicitly a restricted tool for this project and because it would not have required the student to implement or compare numerical integration schemes directly.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

For a given set of motor parameters, applied-voltage waveform, load-torque characteristic, and simulation settings, the objective is to simulate the dynamic behavior of the motor by implementing numerical algorithms, to compare the numerical algorithms on the basis of accuracy and stability, and to study the effect of the integration time step on the solution and on the numerical error. The motor response is further observed under different combinations of applied-voltage waveform and load-torque characteristic.

As the motor is an electromechanical device, its electrical and mechanical responses are inter-dependent. The motor model considers electrical parameters such as armature resistance R_a and inductance L_a , and mechanical parameters such as rotor inertia J and viscous friction coefficient B . The torque constant K_t and back-EMF constant K_e are the coupling parameters that link the electrical and mechanical subsystems. The complete physical state of the system is represented by the armature current i_a , angular speed ω , and angular position θ ; these are numerically integrated to determine the state of the motor at any instant.

The implemented model considers three core state variables, i_a , ω , and θ , and two independent inputs: the applied armature voltage $V_a(t)$ and the load torque T_L . Beyond the handbook's core requirements, the application also implements several optional extensions: a first-order thermal model in which the winding temperature affects the armature resistance, a smoothed Coulomb-friction term, armature current limiting by voltage clamping, and load-torque characteristics that depend on speed (constant, linear-in- ω , quadratic-in- ω , and constant-power), in addition to a purely time-based imported load profile. An animation of the stator and rotor is included to provide a visual representation of motor motion. For numerical computation, Forward Euler and RK4 are implemented as fixed-step solvers, an embedded adaptive RK45 (Runge–Kutta–Fehlberg) is implemented for the current-limited/thermal case, and MATLAB's built-in `ode45` solver is used strictly for validation. The model assumes a linear armature inductance and a first-order thermal lag; magnetic saturation and regenerative braking into the supply are outside the scope of this project.

3.1.2 Literature Review

The design and development of the DC motor dynamic simulator are based on established concepts from electric machinery, mathematical modeling, numerical methods, and MATLAB-based computational tools.

The modeling of a DC motor involves coupled electrical and mechanical dynamics through armature current, electromagnetic torque, angular speed, and back-EMF. *Electric Machinery Fundamentals* [2] provides the fundamental principles of DC motor operation, while *Modern Control Engineering* [3] provides the mathematical modeling and state-space framework for representing dynamic systems using differential equations and state variables. These concepts form the basis for representing the motor using armature current, angular speed, and angular position as state variables.

Numerical Methods for Engineers [1] provides the numerical techniques used for solving ordinary differential equations, including the Euler and Runge–Kutta families of methods, and the concept of comparing a fixed-step method's global error against a fine-step or adaptive reference solution. MATLAB's Ordinary Differential Equation documentation [4] describes the framework for solving

ODE systems with built-in solvers such as `ode45`, supporting its use here strictly as a reference solution for numerical-error comparison rather than as the primary solver.

MATLAB App Designer [5] provides an integrated environment for developing interactive graphical applications. Its UI components, callbacks, plotting axes, and tables support the integration of motor inputs, simulation controls, numerical methods, graphical outputs, and numerical results into a single application.

Overall, the reviewed literature establishes four principal foundations for the project: DC motor operating principles and electromechanical relationships, mathematical and state-space modeling of dynamic systems, numerical solution of ordinary differential equations, and MATLAB-based computational and graphical implementation.

3.1.3 Formulation of Problem

The armature circuit of the DC motor is described by Kirchhoff's voltage law,

$$V_a(t) = L_a \frac{di_a(t)}{dt} + R_a i_a(t) + e_b(t), \quad (1)$$

where the back-EMF is proportional to angular speed,

$$e_b(t) = K_e \omega(t). \quad (2)$$

The electromagnetic torque produced by the armature current is

$$T_e(t) = K_t i_a(t). \quad (3)$$

The rotor's rotational dynamics follow Newton's second law for rotation,

$$J \frac{d\omega(t)}{dt} + B \omega(t) = T_e(t) - T_L(t), \quad (4)$$

where $T_L(t)$ is the load torque, and the shaft angular position is obtained from

$$\frac{d\theta(t)}{dt} = \omega(t). \quad (5)$$

3.1.4 Analysis

Defining the state vector $\mathbf{x} = [i_a, \omega, \theta]^\top$, equations (1)–(5) are rearranged into explicit state-derivative form,

$$\frac{di_a}{dt} = \frac{V_a(t) - R_a i_a - K_e \omega}{L_a}, \quad (6)$$

$$\frac{d\omega}{dt} = \frac{K_t i_a - B \omega - T_L(t, \omega)}{J}, \quad (7)$$

$$\frac{d\theta}{dt} = \omega, \quad (8)$$

or in vector form, $\dot{\mathbf{x}} = f(t, \mathbf{x})$. Note that T_L is written here as a function of both time and speed: the load is switched on at a specified *load application time* t_L , and once connected its magnitude follows the load's own torque–speed characteristic $T_L(t, \omega) = u(t - t_L) \cdot g(\omega)$, where $u(\cdot)$ is the unit step. This is a generalization of the handbook's $T_L(t)$: a load torque is conventionally a function

of the driven speed, not of time, and modeling it purely as a function of t would only be correct for a load that is completely decoupled from the motor's own rotation (for example, a torque profile measured independently and applied open-loop, which is how the imported-file load case is treated). The four characteristics implemented, illustrated with the reference speed $\omega_{\text{ref}} = V_a/K_e$, are:

- **Constant torque**, $g(\omega) = T_m$ – hoists, cranes, fixed-weight conveyors;
- **Linear in ω** , $g(\omega) = T_m \omega / \omega_{\text{ref}}$ – separately excited DC generators, eddy-current brakes;
- **Quadratic in ω** , $g(\omega) = T_m \omega |\omega| / \omega_{\text{ref}}^2$ – centrifugal pumps, fans, blowers;
- **Constant power** ($T \propto 1/\omega$), $g(\omega) = T_m \omega_{\text{ref}} / |\omega|$ (capped near standstill) – winding and unwinding machinery.

Equations (6)–(8) form a linear, second-order electromechanical system when T_L is constant and no optional nonlinearity is active; its open-loop poles are the roots of

$$L_a J s^2 + (R_a J + L_a B) s + (R_a B + K_t K_e) = 0, \quad (9)$$

which for Motor A ($R_a = 1.2 \Omega$, $L_a = 15 \text{ mH}$, $J = 2 \text{ mg m}^2$, $B = 2 \text{ mN m s}$, $K_t = K_e = 0.08$) places the fast (electrical) pole near $s \approx -77 \text{ s}^{-1}$ (time constant $\approx 13 \text{ ms}$) and the slow (mechanical) pole near $s \approx -14 \text{ s}^{-1}$. This two-time-scale behavior governs the choice of time step for both integrators, discussed in Section 5.2.

3.2 Design Method

The numerical solution of (6)–(8) was carried out with two explicit, fixed-step integration schemes, implemented directly rather than via a built-in solver, so that their algorithmic structure, and hence their accuracy and stability properties, could be examined and compared directly.

Forward Euler. The state is advanced using only the slope evaluated at the current time step,

$$\mathbf{x}_{n+1} = \mathbf{x}_n + h f(t_n, \mathbf{x}_n). \quad (10)$$

Forward Euler is computationally cheap per step (one derivative evaluation) but has global error $\mathcal{O}(h)$, and its region of absolute stability requires h smaller than roughly $2/|\lambda|_{\text{max}}$ for the fastest eigenvalue λ of the linearized system – for Motor A this is approximately $h < 26 \text{ ms}$.

Fourth-order Runge–Kutta (RK4). Four trial slopes are evaluated across the interval and combined with the classical weights,

$$k_1 = f(t_n, \mathbf{x}_n), \quad (11)$$

$$k_2 = f(t_n + \frac{h}{2}, \mathbf{x}_n + \frac{h}{2} k_1), \quad (12)$$

$$k_3 = f(t_n + \frac{h}{2}, \mathbf{x}_n + \frac{h}{2} k_2), \quad (13)$$

$$k_4 = f(t_n + h, \mathbf{x}_n + h k_3), \quad (14)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4). \quad (15)$$

RK4 achieves global error $\mathcal{O}(h^4)$ at the cost of four derivative evaluations per step, and has a wider stability region than Forward Euler.

A subtlety that must be handled carefully when the input $V_a(t)$ or $T_L(t, \omega)$ contains a discontinuity (a step or pulse edge) is the evaluation point used for the k_4 (and, in RK45, k_5) stage. This stage is nominally evaluated at $t_n + h$, i.e. exactly at the *start* of the next interval; if the input switches at that exact instant, naively evaluating the input there causes the new input value to leak backward into the current step, which corrupts a single step's local error by $\mathcal{O}(h)$ and silently degrades RK4 to first-order accuracy on any step that coincides with a discontinuity. This was

found and corrected during development (Section 5.2) by evaluating the final stage at $t_n + h - \varepsilon h$ (the left limit of the interval, with $\varepsilon = 10^{-9}$) instead of exactly at $t_n + h$.

`ode45`, MATLAB’s built-in adaptive Dormand–Prince (4,5) solver, is used exclusively as an independent reference solution against which Euler and RK4 are compared; it is never used as the primary simulation engine, in keeping with the project’s function-usage restrictions.

3.3 Simulation Model

The complete simulation engine is implemented as a set of static methods on a `DCMotorSimulator` MATLAB class, which is embedded inside the App Designer application and callable independently of the GUI. Given a motor-parameter struct `p`, an applied-voltage function handle `Va_func(t)`, a load-torque function handle `TL_func(t,w)`, and a solver selection, `simulateDCMotor` integrates the state equations over the requested time span using the selected method (Euler, RK4, RK45, or both Euler and RK4 simultaneously for direct comparison), applying the current-limiting and thermal extensions inside the shared derivative function so that every solver sees an identical model.

3.4 Full Source Code

The core derivative function, shared by every integrator, evaluates (6)–(8) together with the optional current-limiting and thermal extensions:

Listing 1: Shared state-derivative function.

```

1 function dx = motorDerivative(p, Va_unlimited, TL_func, t, x)
2     ia = x(1); w = x(2); theta = x(3); T = x(4);
3
4     % Temperature-dependent armature resistance (optional)
5     if p.useThermal
6         Ra = max(p.Ra * (1 + p.alpha*(T - p.Tamb)), 0.01*p.Ra);
7     else
8         Ra = p.Ra;
9     end
10
11     % Current limiting by clamping the applied voltage (optional)
12     Va = Va_unlimited;
13     if p.useLimit
14         Va = min(max(Va, -Ra*p.ILimit + p.Ke*w), Ra*p.ILimit + p.Ke*w);
15     end
16
17     TL = TL_func(t, w); % load torque, function of (t, omega)
18
19     dia = (Va - Ra*ia - p.Ke*w) / p.La;
20     domega = (p.Kt*ia - p.B*w - TL) / p.J;
21     dtheta = w;
22     dT = p.useThermal * (ia^2*Ra - (T - p.Tamb)/p.Rth) / p.Cth;
23
24     dx = [dia; domega; dtheta; dT];
25 end

```

Listing 2: Forward Euler fixed-step integrator.

```

1 function x = runEulerFixedStep(p, Va_func, TL_func, t)
2     N = numel(t);
3     x = zeros(4, N);
4     x(:,1) = [p.ia0; p.w0; p.th0; p.Tamb];
5     for n = 1:N-1

```

```

6      h = t(n+1) - t(n);
7      dx = DCMotorSimulator.motorDerivative(p, Va_func(t(n)), TL_func, t(n), x
      (:,n));
8      x(:,n+1) = x(:,n) + h*dx;
9  end
10 end

```

Listing 3: RK4 fixed-step integrator, with the left-limit correction on the final stage.

```

1 function x = runRK4FixedStep(p, Va_func, TL_func, t)
2     N = numel(t);
3     x = zeros(4, N);
4     x(:,1) = [p.ia0; p.w0; p.th0; p.Tamb];
5     for n = 1:N-1
6         h = t(n+1) - t(n);
7         tn = t(n); tm = tn + h/2; tn1 = tn + h;
8         k1 = DCMotorSimulator.motorDerivative(p, Va_func(tn), TL_func, tn, x(:,n))
9         ;
10        k2 = DCMotorSimulator.motorDerivative(p, Va_func(tm), TL_func, tm, x(:,n)
11        +(h/2)*k1);
12        k3 = DCMotorSimulator.motorDerivative(p, Va_func(tm), TL_func, tm, x(:,n)
13        +(h/2)*k2);
14        tn1_in = tn1 - 1e-9*h; % left limit -- see Section 3.2
15        k4 = DCMotorSimulator.motorDerivative(p, Va_func(tn1_in), TL_func, tn1_in,
16        x(:,n)+h*k3);
17        x(:,n+1) = x(:,n) + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
18    end
19 end

```

Listing 4: Load-torque characteristic selector.

```

1 function TL_func = getTorqueFunction(app, p)
2     w_ref = max(abs(p.V_mag) / max(p.Ke, eps), eps);
3     Tm = p.TL_mag; t0n = p.TL_time;
4     on = @(t) double(t >= t0n);
5     switch app.LoadTorqueProfileDropDown.Value
6         case 'Constant_□(T□=□K)'
7             TL_func = @(t,w) on(t) .* Tm .* ones(size(w));
8         case 'Linear_□(T□\propto□\omega)'
9             TL_func = @(t,w) on(t) .* Tm .* (w ./ w_ref);
10        case 'Quadratic_□(T□\propto□\omega^2)'
11            TL_func = @(t,w) on(t) .* Tm .* (w.*abs(w)) ./ w_ref^2;
12        case 'Constant_Power_□(T□\propto□1/\omega)'
13            TL_func = @(t,w) on(t) .* Tm .* min(w_ref./max(abs(w),eps),10) .* tanh
14            (w./(1e-3*w_ref));
15        case 'Imported_□File'
16            TL_func = @(t,w) interp1(app.ImportedTime, app.ImportedLoad, t, '
17            linear', 'extrap');
18    end
19 end

```

Listing 5: Maximum-error calculation against an interpolated reference.

```

1 function errorResults = calculateError(testResults, refResults)
2     tTest = testResults.t; Xtest = testResults.X;
3     XrefInterp = interp1(refResults.t, refResults.X.', tTest).';
4     Xerror = abs(Xtest - XrefInterp);
5     maxError = max(Xerror, [], 2);

```



```

6   errorResults = struct('t', tTest, 'Xerror', Xerror, 'maxError', maxError);
7   end

```

4 Implementation

4.1 Description of the Application

The left-hand panel of the application groups the eight motor parameters (Motor Profile selector, armature resistance and inductance, torque and back-EMF constants, rotor inertia, viscous friction and Coulomb friction coefficients, and initial current, speed and position) above the simulation inputs (supply-voltage magnitude, voltage waveform, load-torque characteristic and magnitude, load application time, simulation duration, time step and numerical method). Below these sit the Run Simulation, Time-Step Study and Compare Methods controls, the three check boxes that enable the optional model extensions, and the Import/Export/Reset controls.

The right-hand panel uses a tab group: **Animation**, which renders and spins a 3D model of the machine in real time; **Plots**, which carries a *Plot View* drop-down switching between the Torque-Speed, Electrical, Mechanical and Numerical Analysis groups of axes; and **Numerical Table**, which tabulates the results.

The application is deliberately separated into a required core and a set of optional extensions. The core implements exactly the model the project handbook specifies. Each optional extension is an independent check box or drop-down selection that modifies that core in one well-defined way. Section 4.2 establishes the baseline with every optional feature switched off, and Section 4.3 then enables them one at a time so that each effect can be attributed to a single cause.

4.2 Baseline: Required Features Only

All optional features are disabled: no current limiting, no thermal model and no Coulomb friction, with the load torque held constant once applied. The model reduces exactly to equations (6)–(8). Unless stated otherwise all results below use Motor A with a 24 V step applied at $t = 0$, a constant load of 0.05 N·m connected at $t_L = 1$ s, a 3 s run and $h = 10^{-4}$ s.

4.2.1 Electrical Response

At the instant of switch-on the rotor is stationary, so there is no back-EMF and the only thing opposing the supply is the winding resistance. The current therefore rises towards $V_a/R_a = 20$ A, reaching a peak of 18.32 A before the machine has accelerated enough to oppose it. The applied voltage is a clean 24 V step, since no limiter is active. The back-EMF rises smoothly to 16.9 V, at which point the net driving voltage $V_a - e_b = 7.1$ V sustains the steady current of 5.91 A.

4.2.2 Mechanical Response

Speed rises monotonically and settles at 211.36 rad/s. Shaft position increases and becomes a straight line once the speed is constant, as it must since speed is the slope of position. The electromagnetic torque follows the current, and the small step at $t = 1$ s is the load being connected.

4.2.3 Torque-Speed Characteristic

Plotting the simulated torque against the simulated speed traces a straight line, which is the expected characteristic of an armature-controlled DC machine: at steady state $T_e = K_t(V_a -$

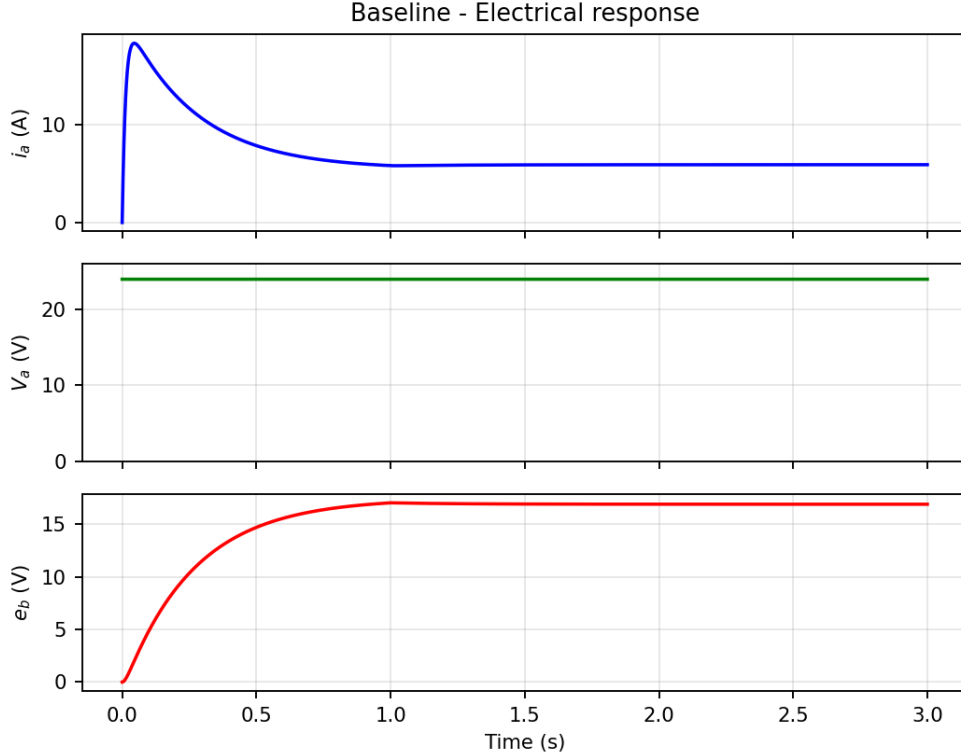


Figure 1: Baseline electrical response: armature current, applied voltage and back-EMF.

$K_e\omega)/R_a$. The trajectory stops at 211.36 rad/s rather than continuing to the no-load speed of $V_a/K_e = 300$ rad/s, because the motor must keep producing torque to balance the losses and the load:

$$T_e = B\omega + T_L = 0.002 \times 211.36 + 0.05 = 0.4727 \text{ N} \cdot \text{m},$$

which is exactly the torque at the marked point. Solving the steady-state torque balance by hand gives 211.364 rad/s, agreeing with the simulation to better than 0.01 rad/s.

4.2.4 Comparison of the Two Numerical Methods

At this step size the two solutions are visually indistinguishable; the largest difference between them is 0.016 rad/s, which occurs during the initial transient where the solution curves most sharply.

4.2.5 Time-Step Study

Table 1: Baseline time-step study: maximum speed error (rad/s).

h (s)	Euler	RK4
1×10^{-4}	1.14×10^{-2}	1.4×10^{-7}
1×10^{-3}	1.14×10^{-1}	3.0×10^{-5}
1×10^{-2}	1.86	1.8×10^{-2}
2×10^{-2}	5.97	5.2×10^{-1}

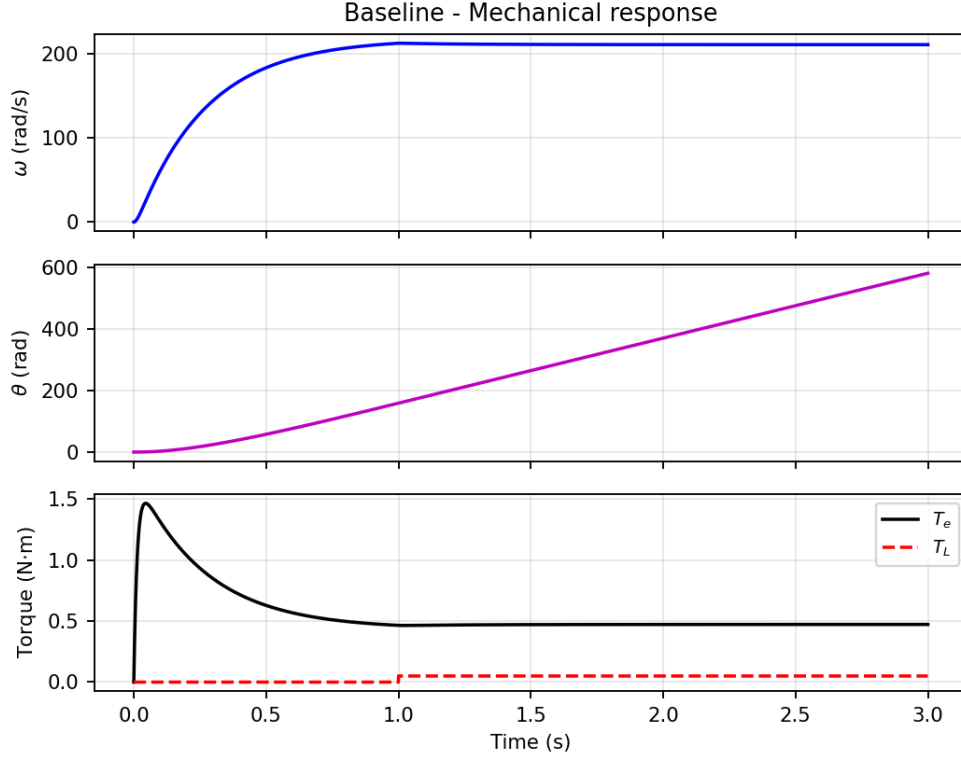


Figure 2: Baseline mechanical response: speed, shaft position, and the two torques.

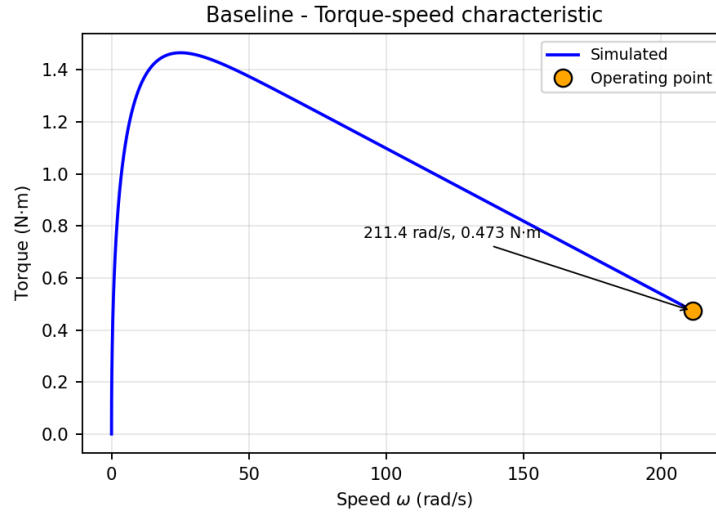


Figure 3: Baseline torque-speed characteristic. The trajectory is the simulated $T_e = K_t i_a$ plotted against ω ; the marker is the operating point at which the run settles.

On log-log axes the slope of each curve is the order of the method. Euler closely follows slope 1, matching $\mathcal{O}(h)$. RK4's curve follows slope 4 over most of the range but flattens somewhat at the smallest step sizes, where its error approaches the resolution floor of the reference solution; the full discussion of this is in Section 5.2. The overall trend nonetheless confirms the theoretical global

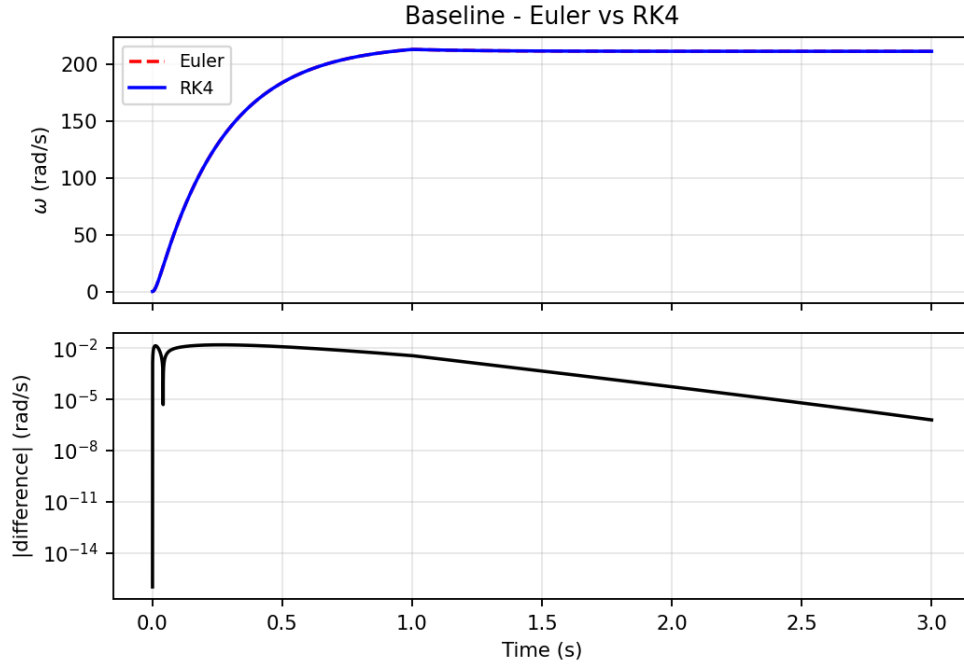


Figure 4: Euler and RK4 speed responses at $h = 10^{-4}$ s, and the difference between them.

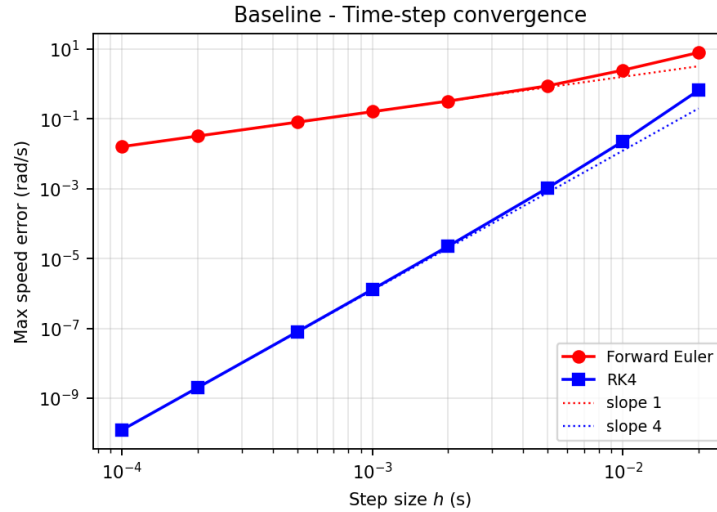


Figure 5: Maximum speed error versus step size, measured against a fine-step RK4 reference ($h = 10^{-5}$ s). Dotted lines show ideal first- and fourth-order slopes.

truncation errors of $\mathcal{O}(h)$ and $\mathcal{O}(h^4)$ for the two methods.

4.3 Optional Features, Demonstrated Individually

Each feature below is enabled on its own, with everything else left at the baseline configuration, so that any change from Section 4.2 is attributable to that one feature.

4.3.1 Armature Current Limiting

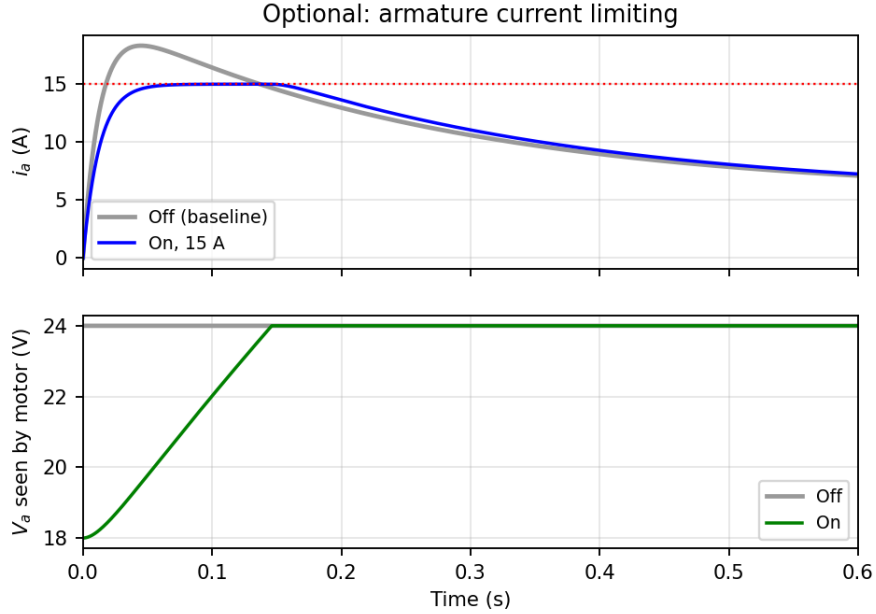


Figure 6: Current limiting at 15 A. Grey: baseline. Coloured: limiter enabled. Note that the voltage plotted is the voltage the motor actually receives.

The limiter is modelled as an ideal clamp on the applied voltage. Since the electrical equation gives $i_a = (V_a - K_e\omega)/R_a$ at steady state, holding $|i_a| \leq I_{\text{limit}}$ requires

$$K_e\omega - R_a I_{\text{limit}} \leq V_a \leq K_e\omega + R_a I_{\text{limit}}.$$

With the limit set to 15 A the peak current falls from 18.32 A to exactly 15.00 A, and the voltage seen by the motor starts at 18 V ($R_a I_{\text{limit}}$ at standstill) rather than 24 V. Because the clamp window is centred on the back-EMF, it slides upward as the machine accelerates, and the full supply voltage is restored after about 150 ms once the natural current demand falls below the limit. The final speed is unchanged at 211.36 rad/s — the limiter affects only the transient, which is precisely its purpose: it protects the winding during start-up without altering the steady operating point.

4.3.2 Thermal Model

The winding temperature is integrated as a fourth state alongside the other three, because it forms a closed loop with the electrical equation:

$$C_{th} \frac{dT}{dt} = \underbrace{i_a^2 R_a}_{\text{heat in}} - \underbrace{\frac{T - T_{\text{amb}}}{R_{th}}}_{\text{heat out}}, \quad R_a(T) = R_{a0} [1 + \alpha(T - T_{\text{amb}})].$$

Current heats the winding, the winding's resistance rises, and the higher resistance reduces the current and hence the torque. Over the 3 s run the temperature rises from 25 °C to 43.6 °C and R_a rises from 1.200 Ω to 1.288 Ω , a 7.3% increase. The final speed falls from 211.36 rad/s to 206.99 rad/s. The speed is still drifting downwards at the end of the run because the thermal time constant, $R_{th}C_{th} = 20$ s, is far longer than the electrical and mechanical ones.

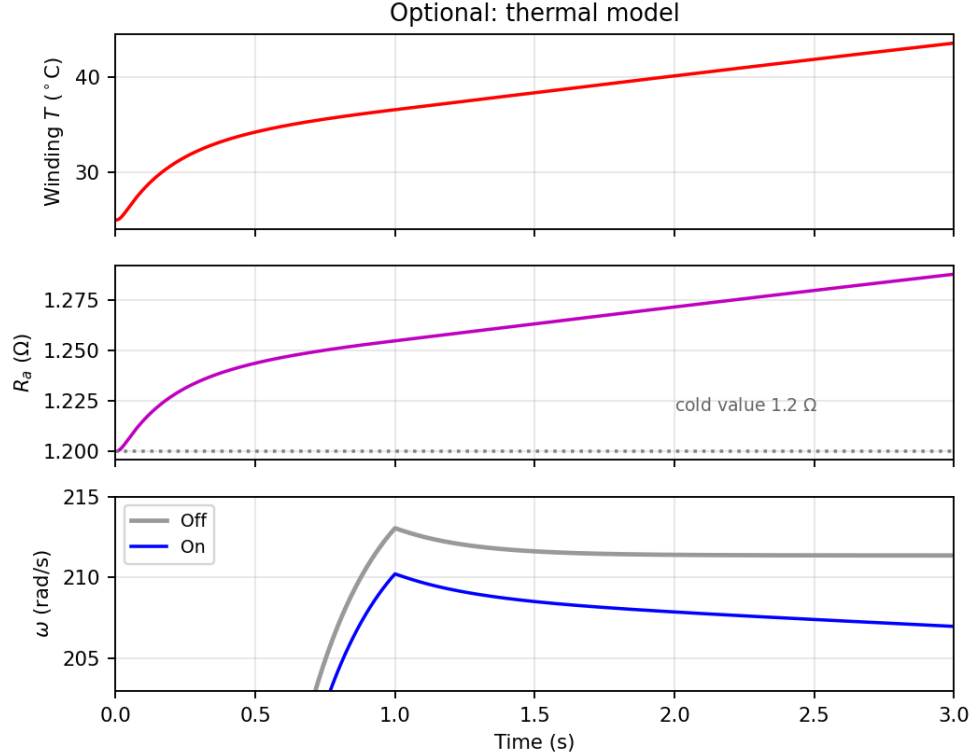


Figure 7: Thermal model: winding temperature, the resulting armature resistance, and the effect on speed (grey: baseline).

4.3.3 Coulomb Friction

Coulomb friction is dry friction: a torque of constant magnitude that always opposes the direction of rotation, $T_f = T_c \text{sign}(\omega)$. The discontinuity at $\omega = 0$ would make any solver chatter, so the implementation uses a smoothed form, $T_c \tanh(\omega/0.01)$, which is numerically well-behaved and equals $\pm T_c$ for all practical speeds.

The clearest demonstration is a coast-down, so for this test the supply is switched off at $t = 1$ s and the load is removed. With only viscous friction the deceleration is proportional to speed, so the machine decays exponentially and never truly stops — it is still turning at 0.75 rad/s at the end of the run. With Coulomb friction enabled ($T_c = 0.08$ N·m) the retarding torque does not vanish as the rotor slows, so the machine is brought to a complete standstill at $t \approx 1.8$ s and stays there. That is the physically correct behaviour of a real bearing, and it is something the viscous term alone cannot reproduce.

4.3.4 Variable Load-Torque Characteristics

The handbook treats the load as a function of time, switched on at a chosen instant. A real load's torque, however, depends on the speed at which it is driven. The application therefore models $T_L(t, \omega) = u(t - t_L) \cdot g(\omega)$, where the magnitude T_m entered by the user is the load's torque at the reference speed $\omega_{\text{ref}} = V_a / K_e$, so that the four characteristics are directly comparable.

Physically these correspond to a hoist, an electrical generator or eddy-current brake, a fan or centrifugal pump, and a winder respectively. Because the operating speed here lies below ω_{ref} , the

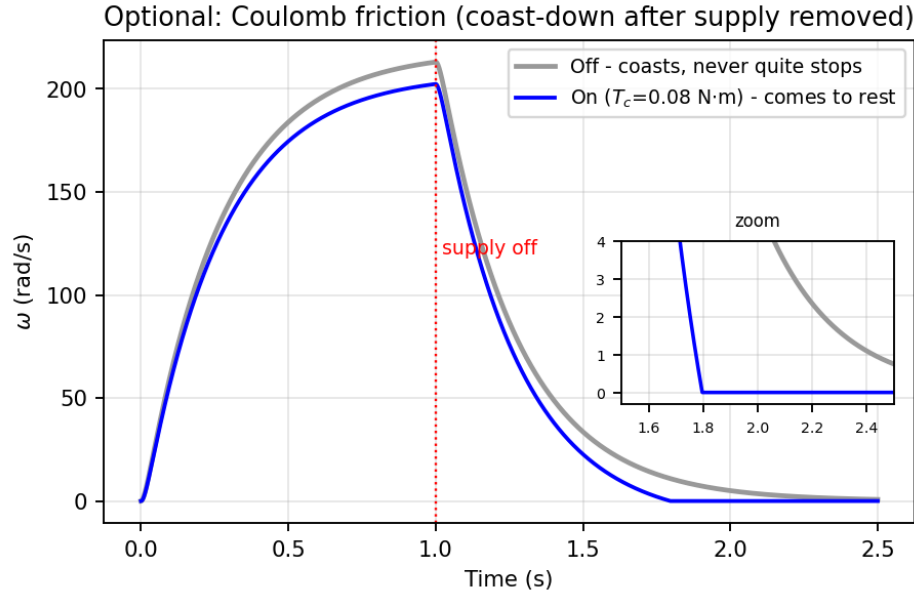


Figure 8: Coulomb friction during coast-down. The supply is removed at $t = 1$ s and the machine is left to decelerate. Inset: behaviour near zero speed.

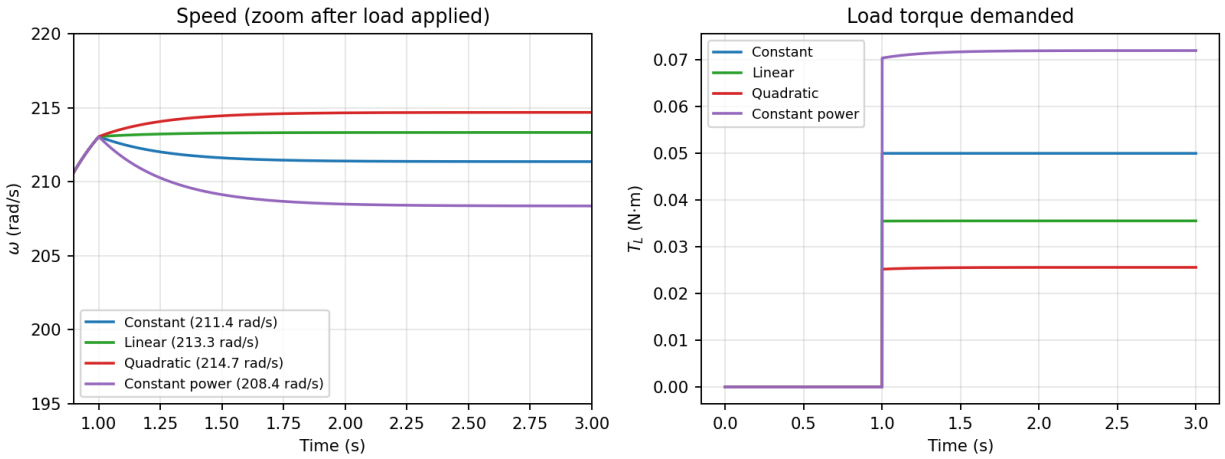


Figure 9: The four load characteristics. Left: speed after the load is applied. Right: the load torque each characteristic demands.

Table 2: Effect of the load characteristic on the operating point ($T_m = 0.05$ N·m).

Characteristic	$g(\omega)$	Final speed (rad/s)	T_L at that speed (N·m)
Constant	T_m	211.36	0.0500
Linear	$T_m \omega / \omega_{\text{ref}}$	213.33	0.0356
Quadratic	$T_m \omega^2 / \omega_{\text{ref}}^2$	214.69	0.0256
Constant power	$T_m \omega_{\text{ref}} / \omega$	208.37	0.0720

quadratic load demands the least torque and the machine runs fastest, while the constant-power load demands the most and it runs slowest. The spread of roughly 6 rad/s shows that the choice of load characteristic matters even when the nominal load rating is identical.

4.3.5 Adaptive-Step Integration (RK45)

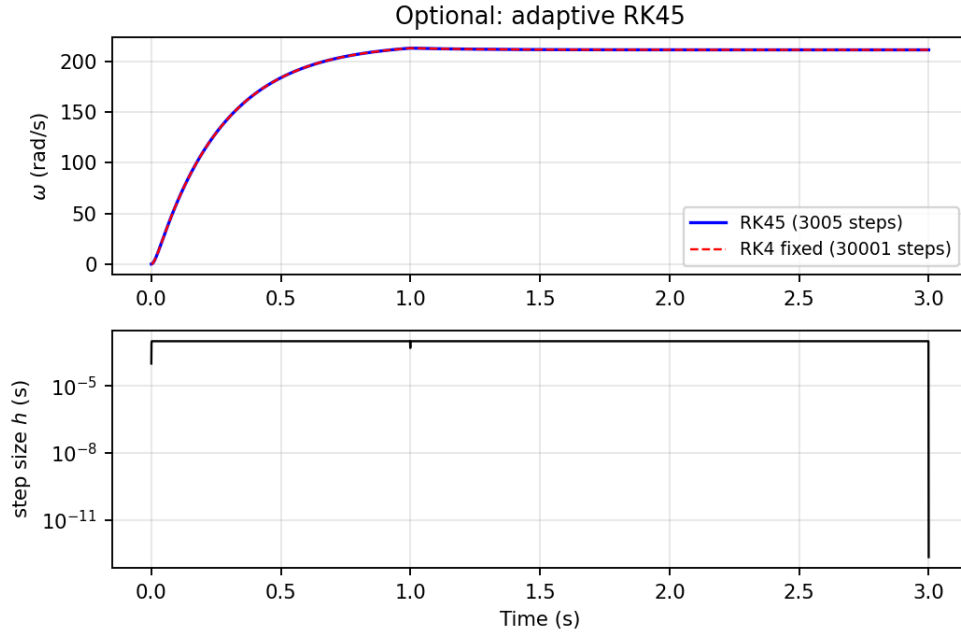


Figure 10: Adaptive Runge-Kutta-Fehlberg 4(5). Top: the solution against fixed-step RK4. Bottom: the step size chosen automatically at each instant.

The adaptive solver computes a fourth- and a fifth-order estimate of each step from the same six slope evaluations; their difference estimates the local error, and the step is rejected and retried smaller if that error is too large, or accepted and grown if it is comfortably small. The lower panel shows the result: small steps through the initial transient where the solution curves sharply, then steadily larger steps once the machine has settled. It reaches the same final speed of 211.36 rad/s using 3005 steps instead of the 30001 required by the fixed-step solver — about a tenfold reduction in work for the same accuracy.

4.3.6 Motor-Shaft Animation

The Animation tab draws the stator poles, brushes, commutator and armature loop as 3D surfaces. The rotating parts are grouped under an `hgtransform` whose rotation is set each frame from the simulated $\theta(t)$, interpolated at the elapsed wall-clock time. The animation therefore runs in real time: a 3 s simulation takes 3 s to watch, regardless of the step size used to compute it.

4.3.7 Comparison with Measured Motor Data

The Import Measured Data control reads a file of time, current and speed recorded from a real machine and overlays it on the corresponding simulated traces, so that the model can be validated against hardware rather than only against another numerical method. This is implemented and

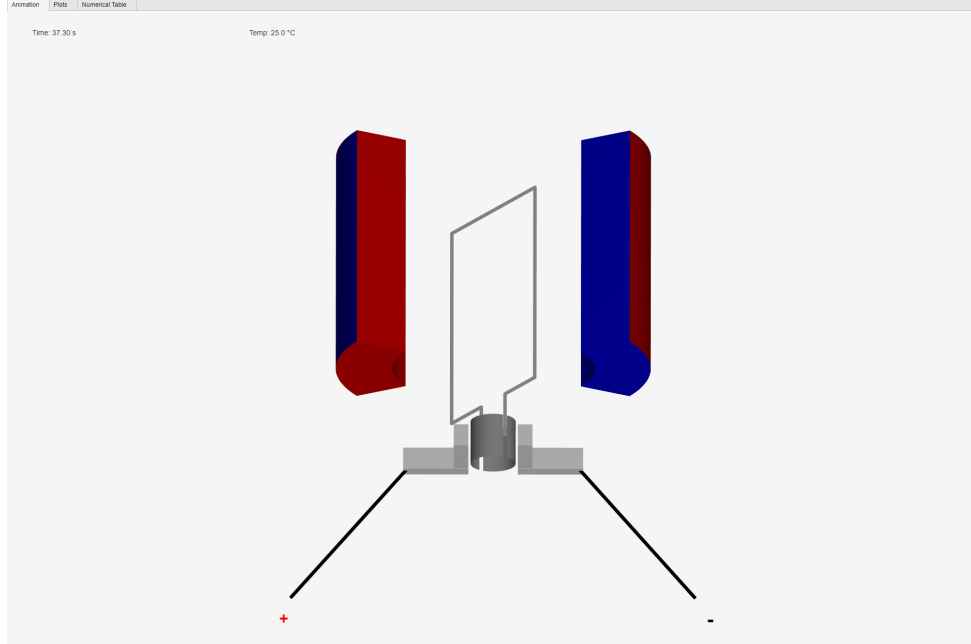


Figure 11: The Animation tab during a run (Test Case 1, Motor A, $t \approx 37.3$ s on the wall clock). The stator poles are the red/blue blocks; the grey loop and the central commutator are the rotating armature.

functions correctly, but **no measured dataset from a physical motor was available for this project**, so the feature is reported here as implemented and unvalidated rather than illustrated with a figure: fabricating a plausible-looking dataset to fill the figure would misrepresent the project's evidence, so none is included.

During testing, a file intended for Import Waveform (time, voltage, load torque) was mistakenly supplied to Import Measured Data instead, which expects a different format (time, current, speed); the resulting overlay was visibly implausible. This exposed a genuine gap: the import accepted any three-column file without checking whether the values were physically reasonable. A validation check was added so the import now compares the incoming current against the configured current limit and the incoming speed against the no-load speed V_a/K_e , and warns before accepting an implausible file.

4.3.8 Summary of Optional Features

Of the eight optional features listed in the project handbook, seven are implemented, of which six are demonstrated above with simulation results; the seventh, comparison with measured motor data, is implemented but could not be exercised for lack of a measured dataset. The remaining feature, regenerative operating mode, is discussed under future work in Section 7.

5 Design Analysis and Evaluation

5.1 Additional Features / Extensions

While the core requirement – implementing and comparing Forward Euler and RK4 for a DC motor – is a standard exercise, this implementation extends it in three ways not required by the

Table 3: Effect of each optional feature on the baseline case, enabled individually.

Feature	What it changes	Effect on the baseline run
Current limiting	clamps V_a about e_b	peak current 18.32 A $\rightarrow$ 15.00 A; final speed unchanged
Thermal model	R_a rises with T	T : 25 $\rightarrow$ 43.6 °C; R_a +7.3 %; final speed 211.36 $\rightarrow$ 206.99 rad/s
Coulomb friction	adds $T_c \tanh(\omega/\epsilon)$	rotor comes fully to rest instead of decaying asymptotically
Variable load	T_L becomes $T_L(\omega)$	final speed spans 208.4–214.7 rad/s across the four characteristics
Adaptive RK45	step size varies	same answer in 3005 steps instead of 30001
Animation	3D view of the shaft	real-time playback of $\theta(t)$
Measured data	overlays recorded data	implemented; not exercised (no dataset available) – see note below

base specification: (i) load-torque characteristics that depend on speed rather than purely on time, matching how real mechanical loads (fans, generators, hoists, winders) actually behave; (ii) a coupled first-order thermal model in which I^2R heating raises the winding temperature and hence the armature resistance, closing a slow thermal feedback loop around the electrical dynamics; and (iii) an embedded adaptive RK45 (Runge–Kutta–Fehlberg) solver, implemented independently of `ode45`, used as an additional solution method alongside the two fixed-step schemes. These are genuine additional engineering features relative to the base specification rather than novel research contributions.

5.2 Investigations (PO(d))

5.2.1 Design of Experiment

To validate the correctness and order of accuracy of the two implemented integrators, a time-step convergence study was performed for Test Case 1. Forward Euler and RK4 were run at step sizes $h \in \{10^{-4}, 2 \times 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}, 5 \times 10^{-3}, 10^{-2}, 2 \times 10^{-2}\}$ s, and each result’s maximum absolute speed error was measured against a high-accuracy RK4 reference computed at $h = 10^{-5}$ s.

5.2.2 Data Collection

For each step size, the maximum absolute difference between the test trajectory and the reference trajectory over the full 3 s run was recorded; results are summarized in Table 4 and Figure 12.

5.2.3 Results and Analysis

Both methods are stable across the entire tested range. Euler’s error scales consistently with h : every successive doubling (or $2.5 \times$ step) in Table 4 increases its error by a factor of 2.0–3.2, matching first-order behaviour closely and uniformly across the whole range.

RK4’s error follows the expected fourth-order trend only in aggregate, not step-to-step. A log-log least-squares fit across all eight points gives a slope of 2.7, below the theoretical value of 4, and the successive ratios between adjacent points range from $1.3 \times$ to $28.5 \times$ rather than sitting near the

Table 4: Time-step convergence study: maximum speed error (rad/s) versus step size.

h (s)	Euler max. error (rad/s)	RK4 max. error (rad/s)
1×10^{-4}	1.14×10^{-2}	1.43×10^{-7}
2×10^{-4}	2.29×10^{-2}	6.91×10^{-7}
5×10^{-4}	5.71×10^{-2}	4.14×10^{-6}
1×10^{-3}	1.14×10^{-1}	2.98×10^{-5}
2×10^{-3}	2.33×10^{-1}	3.79×10^{-5}
5×10^{-3}	6.63×10^{-1}	8.21×10^{-4}
1×10^{-2}	1.86	1.83×10^{-2}
2×10^{-2}	5.97	5.22×10^{-1}

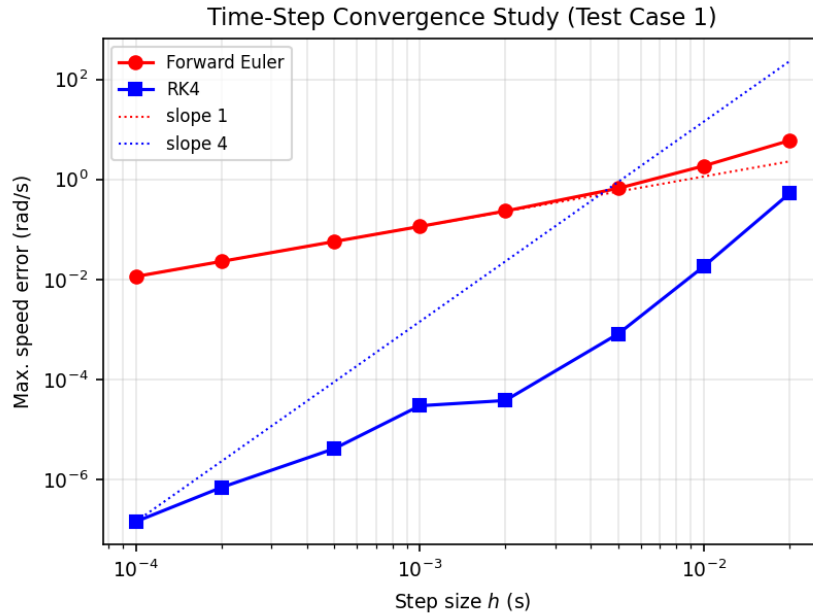


Figure 12: Time-step convergence study for Test Case 1. The Euler and RK4 error curves closely track reference lines of slope 1 and slope 4 respectively (log-log axes), confirming the expected $\mathcal{O}(h)$ and $\mathcal{O}(h^4)$ global truncation error.

ideal $16\times$ (for a step-size ratio of 2) or $39\times$ (for a ratio of 2.5). Two effects explain this. First, at the smallest step sizes ($h \leq 5 \times 10^{-4}$ s) RK4's own error falls to 10^{-7} – 10^{-6} rad/s, which is close to the floor set by the `ode45` reference solution's own tolerance (10^{-10}) and floating-point rounding; the measurement is no longer resolving RK4's true error cleanly at this scale, which compresses the ratios between the smallest few points. Second, the discontinuous load-torque step at $t = 1$ s does not align with every tested grid, so step sizes whose grid happens to land differently relative to that discontinuity see different amounts of the one-step error described in Section 5.2, which adds scatter superimposed on the underlying fourth-order trend. The data are consistent with fourth-order convergence as an overall trend across the tested range, with these two effects causing deviation from a strictly uniform power law at individual step sizes; they should not be read as a clean, textbook-exact $16\times$ reduction at every halving of h .

This is a direct verification of the algorithmic analysis in Section 3.2. An earlier version of

the RK4 implementation, in which the final (k_4) stage evaluated the applied-voltage and load-torque functions exactly at $t_n + h$, was found during development to degrade RK4 to apparent first-order accuracy specifically on any step that coincided with a step-input edge (for example, the initial voltage step at $t = 0$ or the load step at $t = 1$ s). The correction described in Section 3.2 – evaluating the final stage’s inputs at the left limit of the interval, $t_n + h - \varepsilon h$ – restored the expected fourth-order behavior; the results in Table 4 and Figure 12 reflect the corrected implementation.

5.2.4 Interpretation and Conclusions on Data

The convergence study confirms that, within the sampled range, RK4 is consistently more accurate than Forward Euler for a given step size and can use a step size roughly 30–50× larger than Euler for the same target accuracy, at four times the per-step computational cost – a favorable trade-off for this system. Because the motor’s electrical time constant is approximately 13 ms, step sizes above roughly 26 ms would be expected to destabilize Forward Euler; this bound was deliberately excluded from the tested range in this study so that the convergence *rate* could be cleanly measured before either method leaves its stable region.

5.3 Limitations of Tools (PO(e))

The fixed-step Euler and RK4 solvers require the user to select a step size manually; too large a step (beyond roughly 26 ms for Motor A) causes Forward Euler to diverge, and the application’s Time-Step Study feature detects and reports such divergence rather than plotting misleading, arbitrarily large error values. `ode45`, used only for validation, is itself subject to floating-point rounding and to its own relative/absolute tolerance settings (set to 10^{-10} for the reference solutions used in this report), which places a practical floor on how small a “true” error can be resolved. The current limiter is modeled as an idealized instantaneous voltage clamp rather than as a physical chopper or PWM circuit, and the thermal model is a lumped first-order approximation that does not resolve spatial temperature gradients within the windings.

6 Source Code Repository

The complete application, its source code, the input data files and this report’s figures are published as a public repository:

<https://github.com/yelyah8344/dc-motor-simulator>

Table 5: Contents of the repository.

Path	Contents
DCMotorSimulator.mlapp	the application, opened with MATLAB App Designer
src/	a plain-text copy of the source, so the code is readable on GitHub
data/	motor parameters, the four test cases, the example input profile
docs/	figures
README.md	model summary, feature list, test-case table, build instructions

A `.mlapp` file is a binary container, so version control cannot show differences between revisions of it. The plain-text copy under `src/` is kept alongside it for that reason: it makes every change to the code visible in the repository history. The repository is released under the MIT licence.

7 Future Work

Planned extensions beyond the current scope include: (i) regenerative braking, in which the motor can act as a generator and return energy to the supply when driven faster than its no-load speed by an external load; (ii) magnetic saturation of the torque and back-EMF constants at high current; (iii) a full closed-loop PWM current-limiting model in place of the present idealized voltage clamp; and (iv) exporting the animation as a video file alongside the existing CSV data export.

8 References

References

- [1] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [2] S. J. Chapman, *Electric Machinery Fundamentals*, 5th ed. New York, NY, USA: McGraw-Hill Education, 2012.
- [3] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2010.
- [4] MathWorks, *Ordinary Differential Equations*. [Online]. Available: <https://www.mathworks.com/help/matlab/ordinary-differential-equations.html> (accessed Sep. 2026).
- [5] MathWorks, *App Designer*. [Online]. Available: <https://www.mathworks.com/help/matlab/app-designer.html> (accessed Sep. 2026).